LEARN BEING FORWARD

Programming — C++ Programming

Memory management & smart pointers

COMPLETE LEARNING GUIDE

What You'll Learn

C++ gives programmers the same raw access to heap memory that C does through new and delete, but it also provides something C fundamentally lacks: smart pointers, RAII-based wrapper types that automate memory release so that correctly-written modern C++ rarely needs to call delete explicitly at all, or worry about an exception skipping past a cleanup step. This sub-module covers the difference between stack and heap allocation, the specific failure modes of manual new/delete, RAII as the underlying principle smart pointers are built on, and then each of the three standard smart pointer types in turn — unique_ptr for exclusive ownership, shared_ptr for shared, reference-counted ownership, and weak_ptr for breaking the reference cycles shared_ptr alone can create — closing with the modern 'Rule of Zero' philosophy that treats manual memory management as a last resort rather than a default habit.

1. Stack vs. Heap Allocation

Stack-allocated objects (ordinary local variables) are automatically created when their scope begins and automatically destroyed when it ends, with no explicit action required and essentially zero overhead — this automatic lifetime is exactly what RAII and smart pointers build on. Heap-allocated objects (created with new) live until explicitly destroyed with delete, persisting beyond the scope that created them, which is necessary when an object's lifetime must outlive the function that creates it or when its size isn't known until runtime — but that same flexibility means heap memory has no automatic cleanup mechanism at all unless you build one, which is precisely the gap smart pointers fill.

- A local std::vector<int> v; on the stack owns its data on the HEAP internally, but v itself (the handle) is destroyed automatically — this is already a simple form of RAII.
- Stack allocation is fast and automatic but limited in size and scope-bound; heap allocation is flexible in size and lifetime but entirely manual unless wrapped in an RAII type.

2. new/delete and Their Pitfalls

new Type(args) allocates space for one object on the heap and runs its constructor, returning a pointer to it; delete pointer runs the object's destructor and releases the memory. new[] and delete[] are the array equivalents and must always be paired with each other, never mixed with the non-array forms — calling delete (instead of delete[]) on an array-allocated pointer is undefined behavior. Every single new must be matched by exactly one delete: forgetting it leaks memory for the program's remaining lifetime, and calling delete twice on the same pointer (a double free) corrupts the heap's internal bookkeeping, exactly as in C's malloc/free — new/delete don't actually fix any of C's manual-memory hazards, they just add constructor/

destructor calls on top of the same fundamental risk.

```cpp
#include <iostream>

class Widget {
public:
    Widget() { std::cout << "Widget created\n"; }
    ~Widget() { std::cout << "Widget destroyed\n"; }
};

int main() {
    Widget *w = new Widget();  // heap allocation, constructor runs
    delete w;                   // destructor runs, memory released

    int *arr = new int[5];
    delete[] arr;               // array form MUST be used for new[]
    return 0;
}
```

3. RAII Revisited: The Principle Smart Pointers Automate

RAII ties a resource's lifetime to an object's scope: acquire the resource in the constructor, release it in the destructor, and let the language's automatic stack-unwinding guarantee the destructor runs — even if an exception is thrown partway through the enclosing function — so the resource is released exactly once, no matter how control flow exits the scope. Smart pointers are nothing more than small, generic RAII wrapper classes around a raw pointer, implementing exactly this constructor-acquires/destructor-releases pattern so that 'remembering to call delete' stops being a manual discipline you have to get right every single time and becomes something the type system guarantees automatically.

4. unique_ptr: Exclusive Ownership

std::unique_ptr<T> owns a heap object exclusively — exactly one unique_ptr can own a given object at a time, enforced by making unique_ptr non-copyable (copying is a compile error) but movable, so ownership can be explicitly transferred (std::move(ptr)) from one unique_ptr to another, at which point the original becomes empty (nullptr). Its destructor automatically calls delete on the owned object when the unique_ptr itself goes out of scope, which means correctly-written code using unique_ptr for heap ownership never calls delete manually at all. std::make_unique<T>(args) is the preferred way to create one, since it avoids a subtle exception-safety gap that exists with new T(args) passed directly to a unique_ptr constructor in more complex expressions.

- unique_ptr cannot be copied (attempting to do so is a compile error) — only moved, which transfers ownership and empties the source.
- Prefer std::make_unique<T>(args) over raw new T(args) wrapped manually — it's exception-safer and slightly more concise.

```cpp
#include <memory>
#include <iostream>

class Widget {
public:
    Widget() { std::cout << "Widget created\n"; }
    ~Widget() { std::cout << "Widget destroyed\n"; }
    void greet() { std::cout << "Hello from Widget\n"; }
};

int main() {
    std::unique_ptr<Widget> w = std::make_unique<Widget>();
    w->greet();

    std::unique_ptr<Widget> w2 = std::move(w); // ownership transferred; w is now empty
    if (!w) std::cout << "w is now empty\n";
    w2->greet();
    return 0; // w2's destructor automatically deletes the Widget — no manual delete anywhere
}
```

5. shared_ptr and Reference Counting

std::shared_ptr<T> allows multiple shared_ptr instances to jointly own the same heap object, tracked via an internal, atomically-updated reference count — the object is only actually deleted once the LAST shared_ptr owning it is destroyed or reset, making it the right tool when an object's ownership genuinely needs to be shared across multiple parts of a program with no single clear owner. Copying a shared_ptr increments the count; destroying or resetting one decrements it, and the reference-counting overhead (atomic operations, for thread safety) makes shared_ptr measurably more expensive than unique_ptr — it should be reached for only when shared ownership is a genuine requirement, not used as a default 'just in case' choice.

```cpp
#include <memory>
#include <iostream>

class Widget {
public:
    ~Widget() { std::cout << "Widget destroyed\n"; }
};

int main() {
    std::shared_ptr<Widget> a = std::make_shared<Widget>();
    std::cout << "use_count = " << a.use_count() << "\n"; // 1
    {
        std::shared_ptr<Widget> b = a; // shares ownership — count increments
        std::cout << "use_count = " << a.use_count() << "\n"; // 2
    } // b destroyed here — count decrements back to 1
    std::cout << "use_count = " << a.use_count() << "\n"; // 1
    return 0; // a destroyed — count reaches 0, Widget is deleted
}
```

6. weak_ptr and Breaking Circular References

Two objects holding shared_ptrs to each other create a reference cycle where each object's count never reaches zero, because each is still 'owned' by the other — neither is ever destroyed, producing a memory leak that shared_ptr's reference counting cannot detect or resolve on its own. std::weak_ptr<T> observes an object managed by a shared_ptr WITHOUT contributing to its reference count, breaking exactly this kind of cycle — a weak_ptr can't be dereferenced directly (the object might already be gone), so you must call .lock() to obtain a temporary shared_ptr (or an empty one, if the object was already destroyed) before actually using it.

```cpp
#include <memory>
#include <iostream>

struct Child; // forward declaration

struct Parent {
    std::shared_ptr<Child> child;
    ~Parent() { std::cout << "Parent destroyed\n"; }
};

struct Child {
    std::weak_ptr<Parent> parent; // weak_ptr avoids a reference cycle with Parent
    ~Child() { std::cout << "Child destroyed\n"; }
};

int main() {
    auto parent = std::make_shared<Parent>();
    auto child = std::make_shared<Child>();
    parent->child = child;
    child->parent = parent; // weak reference — does NOT increment Parent's use_count
    if (auto p = child->parent.lock()) { // must lock() before use
        std::cout << "Parent is still alive\n";
    }
    return 0; // both Parent and Child are destroyed correctly — no cycle, no leak
}
```

7. The Rule of Zero

The Rule of Zero is the modern C++ philosophy that a well-designed class should need to write NONE of the special member functions (destructor, copy/move constructor, copy/move assignment) itself, by composing it entirely out of members that already manage their own resources correctly — std::string, std::vector, unique_ptr, shared_ptr. When every member already knows how to copy, move, and destroy itself correctly, the compiler-generated versions of the class's own special members are automatically correct too, with zero manual bookkeeping and zero risk of the double-free/leak bugs that plague manual new/delete and raw-pointer-holding classes under the older Rule of Three.

- If you find yourself writing a custom destructor just to call delete on a raw pointer member, replace that raw pointer member with a unique_ptr instead and delete the custom destructor entirely.
- Reach for raw new/delete only in rare, low-level cases (writing a container or smart pointer itself) — application code should essentially never call them directly in modern C++.

Common Pitfalls

- Pairing new[] with delete (instead of delete[]), or new with delete[] — the array and non-array forms must always match.
- Forgetting a delete for every new, leaking memory for the remaining life of the program — exactly as in C, new/delete add no automatic safety by themselves.
- Using a raw pointer after a unique_ptr holding the same object has been moved from or has gone out of scope.
- Creating shared_ptr cycles (two objects each holding a shared_ptr to the other), causing a memory leak reference counting alone cannot detect.
- Dereferencing a weak_ptr directly instead of calling .lock() first and checking whether the returned shared_ptr is non-null.
- Mixing manual new/delete with smart pointers on the same raw resource, or constructing a smart pointer from a pointer that something else also owns/deletes.
- Defaulting to shared_ptr for everything 'just in case' instead of unique_ptr, paying unnecessary reference-counting overhead when exclusive ownership would suffice.

Key Takeaways

- Every new must be matched by exactly one delete (and new[] by delete[]) — smart pointers exist specifically to make this automatic and exception-safe.
- unique_ptr models exclusive ownership: non-copyable, movable, and automatically deletes its object when it goes out of scope.
- shared_ptr models shared ownership via reference counting; the object is deleted only when the last owning shared_ptr is destroyed.
- weak_ptr observes a shared_ptr-managed object without affecting its reference count, specifically to break ownership cycles — call .lock() before use.
- The Rule of Zero: compose classes from members that already manage their own resources (RAII types) so the compiler-generated special members are correct automatically.

Learn More

cppreference — std::unique_ptr
https://en.cppreference.com/w/cpp/memory/unique_ptr

cppreference — std::shared_ptr
https://en.cppreference.com/w/cpp/memory/shared_ptr